

Tax Document Anomaly Detection Platform

Phase 1 Development Reference

Target users: Tax return firms / preparers / reviewers

1. Product Goal

Build a minimal internal platform for tax return firms where a preparer uploads a client ZIP/folder of tax documents, the system extracts and classifies documents, detects missing documents and financial anomalies, prepares a review checklist, and drafts a client email that is sent only after firm approval.

Item	Phase 1 Decision
Product type	B2B tool for tax return firms, not a public tax filing portal
Primary value	Convert messy document packages into a clear anomaly/missing-document checklist
Main user	Firm preparer/reviewer uploads client data and approves email
Automation level	Queue-based processing with human approval before client email
AI role	Helper for classification, account mapping, wording, and ambiguous cases only
Core logic	Deterministic extraction + rule-based anomaly detection first

2. Phase 1 Scope

2.1 Included

- Login screen with basic authenticated access.
- Dashboard listing client processing jobs.
- Upload form: client name, client email, tax year, return type, ZIP file, optional notes.
- Immediate upload acknowledgement and basic file inventory.
- Single queue worker to process one job at a time.
- ZIP extraction, file inventory, document classification, PDF/Excel text extraction.
- Rule-based missing-document and anomaly detection.
- AI-assisted review checklist wording and email draft generation.
- Firm approval before client email is sent.
- Upload additional/missing documents against the same job and re-run verification.

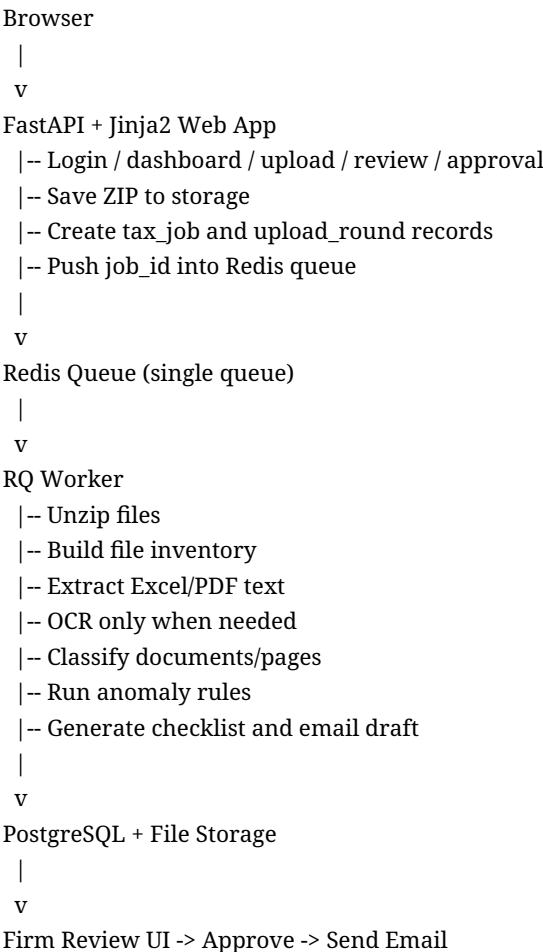
2.2 Excluded From Phase 1

- Direct client portal/login.
- Tax return preparation or IRS filing.
- Full tax advice automation.
- Complex firm billing/multi-tenant SaaS billing.
- Tax software integrations.
- QuickBooks/Xero/bank integrations.
- Fully automated email without firm review.
- Long-term history analytics and advanced reporting.

3. Recommended Technical Stack

Layer	Recommended Tool	Reason
Web app	FastAPI + Jinja2 templates	One Python codebase; enough for minimalist UI
UI styling	Bootstrap 5 or Tailwind CDN	Fast layout without frontend build complexity
Database	PostgreSQL	Reliable relational storage for jobs/checklists/files
ORM/migrations	SQLAlchemy or SQLModel + Alembic	Python-native schema management
Queue	Redis + RQ	Simple Python queue; suitable for single worker Phase 1
File storage	Local disk initially; S3 later	Fast MVP; easy migration later
PDF text	PyMuPDF + pdfplumber	Direct extraction for digital PDFs
Excel	openpyxl / pandas	Accurate extraction without OCR/LLM
OCR fallback	PaddleOCR first; Azure/Textextract later	Lower cost first, paid fallback later
LLM	Gemini Flash or similar low-cost model	Classification/writing helper, not source of truth
Email	SendGrid or SMTP	Approved client request emails

4. High-Level Architecture



5. User Workflow

Step	User/System Action	Output
1	Firm user logs in	Dashboard shown

2	User clicks Upload New Client Job	Upload form displayed
3	User enters client name/email/tax year/return type and uploads ZIP	Job created; ZIP stored
4	System unzips or pre-scans files quickly	Upload acknowledged; basic file count shown
5	System queues job	Job status = QUEUED
6	Worker processes job	Job status = PROCESSING
7	Worker creates checklist/anomalies/email draft	Job status = REVIEW_NEEDED
8	Firm reviews checklist items	Approve, ignore, resolve, or edit items
9	Firm approves email	Email sent to client; job waits for missing docs
10	Firm uploads additional documents to same job	Reprocessing starts and checklist is updated

6. Job Status Lifecycle

Status	Meaning
UPLOADED	Initial ZIP saved and job created
QUEUED	Job added to Redis queue
PROCESSING	Worker is processing ZIP/files
REVIEW_NEEDED	Checklist and anomalies are ready for firm review
EMAIL_DRAFTED	Client email draft has been generated
EMAIL_SENT	Firm approved and email was sent
WAITING_CLIENT_DOCUMENTS	Missing documents requested from client
REPROCESSING	Additional uploaded documents are being processed
COMPLETED	No open high-priority checklist items remain
FAILED	Processing failed; logs should explain reason

7. Minimal UI Screens

Screen	Purpose	Core Fields / Actions
Login	Authenticate firm user	Email, password
Dashboard	List tax jobs	Client, tax year, status, missing count, action
Upload Job	Create new processing job	Client name, email, tax year, return type, ZIP, notes
Job Detail	Show processing result	Status, files, logs, found docs, checklist, email draft
Checklist Review	Approve/edit findings	Approve, ignore, mark resolved, edit request text
Email Review	Approve and send client email	Subject, body, selected checklist items, send button
Upload Additional Docs	Attach missing/proof documents	Upload ZIP/files to existing job and re-run

8. Database Model

For Phase 1, keep the model lean. Store anomalies and missing document requests together as checklist items to avoid over-engineering.

```

users
- id
- name
- email

```

- password_hash
- role
- created_at

tax_jobs

- id
- client_name
- client_email
- tax_year
- return_type
- status
- uploaded_zip_path
- notes
- created_by
- created_at
- updated_at

upload_rounds

- id
- tax_job_id
- round_number
- upload_type # INITIAL / MISSING_DOCUMENTS
- zip_path
- status
- created_at

job_files

- id
- tax_job_id
- upload_round_id
- file_name
- file_path
- file_type
- detected_document_type
- page_count
- extraction_method
- confidence_score
- is_processed
- created_at

checklist_items

- id
- tax_job_id
- type # MISSING_DOCUMENT / ANOMALY / WARNING
- title
- description
- severity # HIGH / MEDIUM / LOW
- status # OPEN / APPROVED / IGNORED / RESOLVED
- source_file
- source_page
- recommended_document
- email_text

- created_at

email_requests

- id
- tax_job_id
- email_to
- subject
- body
- status # DRAFT / APPROVED / SENT / FAILED
- approved_by
- sent_at
- created_at

processing_logs

- id
- tax_job_id
- level # INFO / WARNING / ERROR
- message
- created_at

9. Suggested Project Structure

tax_ai_app/
 app/
 main.py
 config.py
 database.py

 models/
 user.py
 tax_job.py
 upload_round.py
 job_file.py
 checklist_item.py
 email_request.py
 processing_log.py

 routes/
 auth.py
 dashboard.py
 jobs.py
 checklist.py
 email.py

 services/
 zip_service.py
 file_inventory_service.py
 pdf_service.py
 excel_service.py
 ocr_service.py
 document_classifier.py

anomaly_engine.py
llm_service.py
email_service.py
queue_service.py

worker/
processor.py
tasks.py

templates/
base.html
login.html
dashboard.html
upload_job.html
job_detail.html
checklist.html
email_review.html

static/
css/
js/

storage/
jobs/
alembic/
requirements.txt
docker-compose.yml

10. Extraction Pipeline

The extraction router must choose the cheapest reliable method first. Do not send everything to OCR or LLM.

Input Type	Primary Method	Fallback Method	LLM Use
Excel / CSV	openpyxl / pandas	Manual review if malformed	No LLM needed
Digital PDF	PyMuPDF / pdfplumber	Table extraction with pdfplumber/Camelot	Only if classification unclear
Scanned PDF	PaddleOCR	Azure Document Intelligence or AWS Textract	Only for failed/low-confidence pages
Image receipt/proof	PaddleOCR	Cloud OCR	Short summary/classification only
Unknown document	Filename + text heuristic	LLM classification using short snippets	Yes, classify only

Extraction routing logic:

```
if file is Excel/CSV:
    parse directly with openpyxl/pandas
elif PDF has selectable text:
    extract with PyMuPDF/pdfplumber
    extract tables if needed
elif PDF/image is scanned:
    OCR with PaddleOCR
    if confidence is low:
```

send selected pages to paid OCR or Gemini Flash vision
else:
mark UNKNOWN and create manual review checklist item

11. AI / LLM Usage Strategy

Use AI as a reviewer/helper, not as the source of truth for numbers. This keeps cost lower and reduces hallucination risk.

Use AI For	Avoid Using AI For
Classifying unclear documents from snippets	Reading every page of every ZIP
Mapping unusual account names to standard categories	Extracting full trial balances line by line
Explaining rule-based anomalies in friendly language	Performing balance sheet math
Generating email draft from approved checklist	Deciding final tax treatment
Cleaning messy OCR output for selected pages	Replacing deterministic parsers

11.1 Gemini Flash Usage

Gemini Flash or a similar low-cost model can be useful if used on small inputs. Recommended usage: document/page classification, account normalization, checklist wording, and email generation. Avoid sending full large PDFs or entire ZIP contents to the model.

Cost control rules:

1. Never call LLM for Excel files.
2. Never call LLM if PDF direct extraction confidence is high.
3. Only send snippets, not full documents, for classification.
4. For scanned pages, OCR locally first; use Gemini only on selected failed pages.
5. Cache classification results by file hash.
6. Store extraction_method and confidence_score for every extracted value.

12. Phase 1 Document Types

Document Type	Detection Clues	Common Use
PRIOR_YEAR_RETURN	Form 1120-S, 1065, 1120, 1040 Schedule C, prior year	Compare prior-year values and entity type
BALANCE_SHEET	Assets, liabilities, equity, retained earnings	Balance sheet equation and missing support
INCOME_STATEMENT	Revenue, expenses, net income	Expense-driven missing document rules
TRIAL_BALANCE	Debit, credit, account number, ending balance	Account-level validation
GENERAL_LEDGER	Transaction detail by account	Support adjustments and unusual balances
BANK_STATEMENT	Bank name, statement period, deposits, withdrawals	Cash support
PAYROLL_SUMMARY	Wages, payroll taxes, employees	Payroll support
W2_W3	W-2, W-3, employee wage statements	Payroll tax support
FORM_1099	1099-NEC, 1099-K, 1099-MISC	Income/contractor support
LOAN_STATEMENT	Principal, interest, loan balance	Loan/interest support
DEPRECIATION_SCHEDULE	Assets, depreciation, accumulated depreciation	Fixed asset support
UNKNOWN	Low confidence	Manual review

13. Phase 1 Anomaly and Missing Document Rules

Start with a small, explainable rule set. Each rule should create a checklist item with severity, reason, recommended document, and email wording.

Rule	Condition	Checklist Item
Missing prior return	No prior year tax return detected	Request prior year return
Missing balance sheet	No balance sheet detected	Request current year balance sheet
Missing income statement	No income statement detected	Request current year income statement / P&L
Missing trial balance	Balance sheet and income statement found but trial balance not found	Request current year trial balance
Missing general ledger	Adjusted/unadjusted statements or journal entries detected but GL not found	Request general ledger detail
Balance sheet mismatch	Total assets != liabilities + equity	Request corrected/adjusted balance sheet or trial balance
Payroll support missing	Payroll/wages expense exists but payroll docs missing	Request payroll summary, W-2/W-3, payroll filings
Depreciation support missing	Depreciation expense or fixed assets detected but schedule missing	Request fixed asset/depreciation schedule
Loan support missing	Loan payable or interest expense exists but loan statement missing	Request loan statement and interest support
AR support missing	Accounts receivable balance exists but AR aging missing	Request AR aging report
AP support missing	Accounts payable balance exists but AP aging missing	Request AP aging report
Bank support missing	Cash/bank balance exists but bank statement/reconciliation missing	Request December bank statement and reconciliation
High miscellaneous expense	Miscellaneous expense exceeds configured threshold	Request breakdown and supporting detail
Shareholder distribution support	Shareholder distributions/draws/cash advance detected	Request distribution/shareholder advance detail
Negative balance warning	Negative asset/liability values found	Request reclassification or explanation

Example rule output:

```
{
  "type": "MISSING_DOCUMENT",
  "title": "Trial Balance Missing",
  "severity": "HIGH",
  "description": "Balance sheet and income statement were found, but no account-level trial balance was detected.",
  "recommended_document": "Current year trial balance",
  "email_text": "Please provide the current year trial balance so we can verify account-level balances and classifications."
}
```

14. Queue Worker Processing Algorithm

```
def process_tax_job(job_id):
    mark_job_status(job_id, "PROCESSING")

    upload_rounds = get_unprocessed_upload_rounds(job_id)

    for round in upload_rounds:
        unzip_files(round.zip_path)
        create_file_inventory(job_id, round)
        classify_files(job_id, round)
```



```

extract_text_and_tables(job_id, round)
normalize_financial_data(job_id, round)

run_document_presence_rules(job_id)
run_financial_anomaly_rules(job_id)
match_new_files_to_open_checklist_items(job_id)
generate_or_update_email_draft(job_id)

if has_open_items(job_id):
    mark_job_status(job_id, "REVIEW_NEEDED")
else:
    mark_job_status(job_id, "COMPLETED")

```

15. Additional Document Reprocessing

Additional documents should be uploaded against the same tax job, not as a new job. Create a new `upload_round` and process only new files. Then match newly found document types against unresolved checklist items.

Additional document flow:

1. User opens existing job.
2. User clicks Upload Additional Documents.
3. System creates `upload_round` with `upload_type = MISSING_DOCUMENTS`.
4. System queues same `tax_job_id`.
5. Worker processes only new `upload_round` files.
6. Worker updates `job_files` and extracted data.
7. Worker marks matching checklist items `RESOLVED` if evidence found.
8. Worker keeps unresolved items `OPEN`.
9. Status becomes `REVIEW_NEEDED` or `COMPLETED`.

16. Minimal Routes / APIs

Route	Method	Purpose
/login	GET	Show login page
/login	POST	Authenticate user
/logout	POST	End session
/dashboard	GET	List tax jobs
/jobs/new	GET	Show upload form
/jobs	POST	Create job and upload ZIP
/jobs/{id}	GET	Show job detail
/jobs/{id}/additional-documents	POST	Upload missing/additional documents
/jobs/{id}/checklist	GET	Show checklist
/checklist-items/{id}	POST/PATCH	Approve/ignore/resolve/edit item
/jobs/{id}/email	GET	Show email draft
/jobs/{id}/email/send	POST	Send approved email

17. Docker Compose Draft

```

services:
  web:
    build: .

```

command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

ports:

- "8000:8000"

env_file:

- .env

volumes:

- ./app

- ./storage:/app/storage

depends_on:

- postgres

- redis

worker:

build: .

command: rq worker tax_jobs --url redis://redis:6379/0

env_file:

- .env

volumes:

- ./app

- ./storage:/app/storage

depends_on:

- postgres

- redis

postgres:

image: postgres:16

environment:

POSTGRES_DB: tax_ai

POSTGRES_USER: tax_ai

POSTGRES_PASSWORD: tax_ai_password

ports:

- "5432:5432"

volumes:

- postgres-data:/var/lib/postgresql/data

redis:

image: redis:7-alpine

ports:

- "6379:6379"

volumes:

postgres-data:

18. Security and Compliance Notes

- Treat all uploaded tax documents as sensitive financial data.
- Use HTTPS in all environments beyond local development.
- Store passwords with a strong password hash such as bcrypt/argon2.
- Restrict uploaded file types and maximum ZIP size.
- Scan ZIP paths to prevent zip-slip path traversal attacks.
- Keep files separated by job_id and firm/user scope.
- Log processing events but avoid logging SSNs, EINs, bank account numbers, or full document text.

- Do not send client emails automatically; require firm approval.
- Add a disclaimer that the platform assists with document readiness and does not provide tax/legal/accounting advice.

19. Suggested Development Milestones

Milestone	Deliverable
M1 - App skeleton	FastAPI, Jinja2 layout, login, dashboard, database connection
M2 - Job upload	Create job, upload ZIP, store file, create upload_round
M3 - Queue worker	Redis/RQ setup, worker picks job and updates status
M4 - File inventory	Unzip safely, list files, detect extensions and page counts
M5 - Basic extraction	Excel parser, PyMuPDF/pdfplumber text extraction
M6 - Classification	Document type detection via filename + text rules; LLM fallback for unknowns
M7 - Rule engine	Generate initial missing-document/anomaly checklist
M8 - Review UI	View checklist, approve/ignore/resolve/edit items
M9 - Email draft/send	Generate email from approved items and send via SendGrid/SMTP
M10 - Additional docs	Upload new ZIP/files to existing job and re-run verification
M11 - Hardening	Error handling, processing logs, file size limits, security checks

20. Phase 1 Acceptance Criteria

- Firm user can log in and see dashboard.
- Firm user can create a client tax job and upload a ZIP.
- System stores ZIP, unzips safely, and shows files found.
- Queue worker processes one job at a time and updates job status.
- System detects at least core documents: prior return, balance sheet, income statement, trial balance, general ledger, payroll docs, bank statement, depreciation schedule.
- System creates checklist items for missing trial balance/general ledger and common financial support documents.
- Firm user can approve, ignore, or resolve checklist items.
- System generates an email draft from approved checklist items.
- Firm user can send email after approval.
- Firm user can upload additional documents to the same job and re-run verification.
- System does not require LLM for deterministic parsing or calculations.

21. Final Development Principle

For Phase 1, the strongest feature is not a beautiful UI or full automation. The strongest feature is reliable conversion of a messy ZIP into a clean document inventory, anomaly checklist, and approved client request email. Keep extraction deterministic where possible, keep AI usage controlled, and keep the tax firm in the approval loop.